

GTK3 Wrapper Library

Filière d'ingénieur:

**Ingénierie Logicielle et Intégration des
Systèmes Informatiques**

Encadré par :

- **Pr. BEKHOUCHA Abderrahim**

Réalisé par :

SAAD AIT YAHIA

Ali EZZOUHDI

OUSSAMA EL-JARRARI

ERRAMI YOUSSE

GTK3 Wrapper Library

Table of Contents

Contents

- Pr. BEKHOUCHA Abderrahim	1
 Table of Contents	2
Project Overview	4
Key Features	4
Architecture	4
Core Files Documentation	5
 structures.h - Header File	5
 structures.c - Implementation File	6
Widget Reference	7
1. Window (window)	7
2. Button (button)	8
3. Label (label)	8
4. Entry (Text Input)	8
5. Box (Layout Container)	9
6. Radio Button	9
7. Check Button	10
8. Progress Bar	10
9. Spinner	10
10. Image	10
11. Menu System	11
Examples	11
Example 1: Hello World	11
Example 2: Form with Validation	12
Installation & Compilation Guide	13
Prerequisites	13
Setting Up the Compiler Script	14
Compiling Your Program	15
Project Structure	16

Running on WSL Ubuntu	16
Troubleshooting.....	16
Common Issues and Solutions	16
Debugging Tips.....	17
Best Practices.....	17
API Quick Reference	17
Core Functions.....	17
Window Functions.....	18
Widget Creation.....	18
Layout Functions.....	18
Utility Functions.....	18

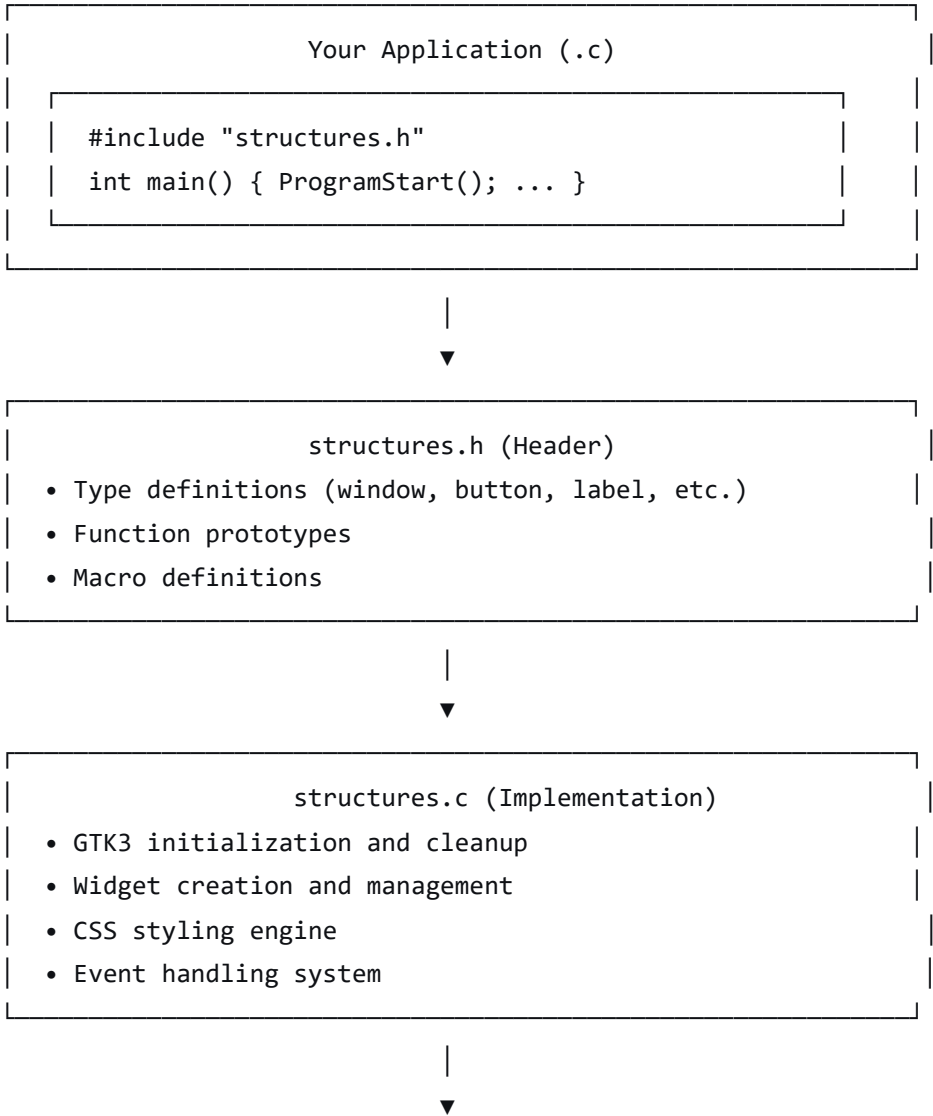
Project Overview

This project is a **C wrapper library for GTK3** (GIMP Toolkit version 3) that simplifies GUI application development. It provides an intuitive, object-oriented-like API for creating graphical interfaces without needing to understand the complexity of raw GTK3 function calls.

Key Features

Feature	Description
Cross-platform	Works on Linux, WSL, and other Unix-like systems
Simplified API	Easy-to-use functions for common GUI widgets
CSS Styling	Built-in support for custom styling
Memory Safety	Automatic widget memory management
Event-driven	Simple callback system for user interactions

Architecture



GTK3 Library

- Windowing system abstraction
- Native OS rendering

Core Files Documentation

■ structures.h - Header File

This file defines all data structures and function prototypes. It's the **interface** your programs will interact with.

Main Data Structures

// Window structure - represents the main application window

```
typedef struct win {
    GtkWidget *win;           // Internal GTK widget
    char *title;              // Window title
    char *icon_path;          // Path to window icon
    char *bg_image;           // Background image path
    char *bg_color;           // Background color (hex: "#RRGGBB")
    int width, height;        // Window dimensions
    gboolean resizable;        // Can user resize?
    gboolean modal;           // Block other windows?
    double opacity;           // Transparency (0.0 to 1.0)
    gboolean fullscreen;      // Fullscreen mode?
    GtkWindowType type_win;   // Window type
} window;
```

// Button structure

```
typedef struct btn {
    GtkWidget *button;        // Internal GTK widget
    char *label;              // Button text
    char *bg_color;           // Background color
    char *label_style;        // "bold", "italic", etc.
    int width, height;        // Button size
    void (*click_callback)(GtkWidget*, gpointer); // Click handler
    void *callback_data;      // Data passed to callback
} button;
```

// Box structure - for arranging widgets

```
typedef struct bx {
    GtkWidget *box;           // Internal GTK widget
```

```
GtkOrientation orientation; // GTK_ORIENTATION_HORIZONTAL or VERTICAL
int spacing;                // Space between children
int margin_top, margin_bottom, margin_left, margin_right;
gboolean homogeneous;      // Equal size for all children?
} box;
```

Core Function Categories

Category	Functions
Lifecycle	ProgramStart(), MainStart(), ProgramEnd()
Window	create_window(), update_window(), display_window(), close_window()
Widgets	create_button(), create_label(), create_entry(), etc.
Layout	create_hbox(), create_vbox(), add_to_box()
Styling	apply_css()
Menu	create_horizontal_menu(), add_menu_item(), add_submenu()

■ structures.c - Implementation File

This file contains the actual GTK3 code that makes everything work. It's the **engine** of the library.

Key Implementation Details

1. GTK Initialization & Main Loop

```
void ProgramStart() {
    gtk_init(NULL, NULL);           // Initialize GTK
    global_css_provider = gtk_css_provider_new(); // CSS support
}

void MainStart() {
    gtk_main();                     // Start the event loop
}

void ProgramEnd() {
    gtk_main_quit();                // Exit the event loop
}
```

2. CSS Styling Engine

```
void apply_css(GtkWidget *widget, const char *css) {
    GtkCssProvider *provider = gtk_css_provider_new();
    gtk_css_provider_load_from_data(provider, css, -1, NULL);
    GtkStyleContext *context = gtk_widget_get_style_context(widget);
    gtk_style_context_add_provider(context, GTK_STYLE_PROVIDER(provider),
                                  GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
}
```

```

    g_object_unref(provider);
}

```

This function allows dynamic styling of any widget using CSS syntax.

3. Radio Button Group Management

The library implements a custom radio button group system using a linked list:

```

typedef struct RadioGroupNode {
    char *group_id;           // Group identifier (e.g., "colors", "sizes")
    GSList *group;           // GTK's radio button group
    struct RadioGroupNode *next;
} RadioGroupNode;

static RadioGroupNode *radio_groups = NULL; // Global group registry

```

4. Widget Creation Pattern

Every widget follows the same pattern:

1. **Allocate** memory for the structure
2. **Create** the underlying GTK widget
3. **Set defaults** for properties
4. **Return** the structure pointer

Example for button:

```

button* create_button() {
    button *btn = malloc(sizeof(button));
    btn->button = gtk_button_new();
    btn->label = g_strdup("Button");
    btn->width = -1;           // -1 means "use default"
    btn->height = -1;
    btn->click_callback = NULL;
    // ... set other defaults
    return btn;
}

```

Widget Reference

1. Window (window)

```

window *win = create_window();
win->title = "My App";

```

```

win->width = 800;
win->height = 600;
win->bg_color = "#f0f0f0";
win->resizable = TRUE;
update_window(win);
add_to_window(win, some_widget);
display_window(win);
close_window(win); // Connect destroy signal

```

2. Button (button)

```

void on_click(GtkWidget *w, gpointer data) {
    printf("Button clicked! Data: %s\n", (char*)data);
}

```

```

button *btn = create_button();
btn->label = "Click Me";
btn->bg_color = "#4CAF50"; // Green
btn->width = 120;
btn->height = 40;
btn->click_callback = on_click;
btn->callback_data = "Hello";
update_button(btn);

```

3. Label (label)

```

label *lbl = create_label();
lbl->text = "Welcome!";
lbl->color = "#333333";
lbl->bg_color = "#eeeeee";
lbl->style = "bold italic";
lbl->position = "center"; // Left, center, right
update_label(lbl);

```

4. Entry (Text Input)

```

void on_changed(GtkWidget *w, gpointer data) {
    entry *en = (entry*)data;
    printf("Text: %s\n", get_entry_text(en));
}

```

```

entry *en = create_entry();
en->placeholder_text = "Enter text...";
en->width = 300;
en->height = 35;

```



```

en->visibility = TRUE;    // FALSE for password mode
en->changed_callback = on_changed;
en->callback_data = en;
update_entry(en);

// Get/Set text
char *text = get_entry_text(en);
set_entry_text(en, "New text");

```

5. Box (Layout Container)

```

// Vertical box (stacks widgets top to bottom)
box *vbox = create_vbox(spacing, homogeneous);
// spacing = pixels between children
// homogeneous = TRUE makes all children same size

// Horizontal box (arranges left to right)
box *hbox = create_hbox(10, FALSE);

// Configure box
vbox->margin_top = 20;
vbox->margin_bottom = 20;
vbox->margin_left = 10;
vbox->margin_right = 10;
vbox->bg_color = "#ffffff";
update_box(vbox);

// Add widgets
add_to_box(vbox, widget);
add_to_box_with_properties(vbox, widget, expand, fill, padding);

```

6. Radio Button

```

void on_toggled(GtkWidget *w, gpointer data) {
    radio_button *rdb = (radio_button*)data;
    if (is_radio_button_active(rdb)) {
        printf("Selected: %s\n", rdb->label);
    }
}

// Buttons with same group_id are mutually exclusive
radio_button *red = create_radio_button("colors", "Red");
red->toggled_callback = on_toggled;
red->callback_data = red;
red->active = TRUE;    // Default selection
update_radio_button(red);

```

```
radio_button *green = create_radio_button("colors", "Green");
green->toggled_callback = on_toggled;
update_radio_button(green);
```

7. Check Button

```
check_button *chk = create_check_button();
chk->label = "Enable feature";
chk->active = FALSE;
chk->toggled_callback = on_check_toggled;
update_check_button(chk);

gboolean is_checked = is_check_button_active(chk);
```

8. Progress Bar

```
progress_bar *pb = create_progress_bar();
pb->width = 300;
pb->height = 25;
pb->show_text = TRUE;    // Show percentage
update_progress_bar(pb);

// Set value (0.0 to 1.0)
set_progress_value(pb, 0.75); // 75%

// Pulsing mode (for indeterminate progress)
start_progress_pulse(pb);
stop_progress_pulse(pb);
```

9. Spinner

```
spinner *spin = create_spinner();
spin->text = "Loading...";
update_spinner(spin);
add_to_box(box, get_spinner_widget(spin));

start_spinner(spin);
stop_spinner(spin);
```

10. Image

```
image *img = create_image();
img->file_name = "photo.jpg";
img->width = 200;
```

```
img->height = 150;
img->position = "center";
img->border_width = 2;
img->border_color = "#cccccc";
update_image(img);
```

11. Menu System

```
// Create menu bar
menu *menubar = create_horizontal_menu();

// Create submenu
menu *file_menu = create_submenu();
add_menu_item(file_menu, "New", on_new, NULL);
add_menu_item(file_menu, "Open", on_open, "file.txt");
add_menu_separator(file_menu);
add_menu_item(file_menu, "Quit", on_quit, NULL);

// Add to menu bar
add_submenu(menubar, "File", file_menu);

// Menu button (dropdown from a button)
menu_button *mb = create_menu_button("Menu");
set_menu_button_menu(mb, file_menu);
update_menu_button(mb);
```

Examples

Example 1: Hello World

```
#include "structures.h"

int main() {
    ProgramStart();

    window *win = create_window();
    win->title = "Hello World";
    win->width = 300;
    win->height = 200;
    update_window(win);

    label *lbl = create_label();
    lbl->text = "Hello, GTK!";
    lbl->style = "bold";
```

```

    update_label(lbl);

    add_to_window(win, lbl->label);
    display_window(win);
    close_window(win);

    MainStart();
    ProgramEnd();
    return 0;
}

```

Example 2: Form with Validation

```

#include "structures.h"

window *win;
entry *name_entry, *email_entry;
label *result;

void on_submit(GtkWidget *w, gpointer data) {
    char *name = get_entry_text(name_entry);
    char *email = get_entry_text(email_entry);
    char msg[256];

    if (strlen(name) > 0 && strlen(email) > 0) {
        snprintf(msg, sizeof(msg), "Submitted: %s (%s)", name, email);
    } else {
        snprintf(msg, sizeof(msg), "Please fill all fields");
    }
    gtk_label_set_text(GTK_LABEL(result->label), msg);
}

int main() {
    ProgramStart();

    win = create_window();
    win->title = "Contact Form";
    win->width = 400;
    win->height = 300;
    update_window(win);

    box *vbox = create_vbox(10, FALSE);
    vbox->margin = 20;
    update_box(vbox);

    name_entry = create_entry();

```

```

name_entry->placeholder_text = "Your name";
update_entry(name_entry);

email_entry = create_entry();
email_entry->placeholder_text = "Email address";
update_entry(email_entry);

button *submit = create_button();
submit->label = "Submit";
submit->bg_color = "#4CAF50";
submit->click_callback = on_submit;
update_button(submit);

result = create_label();
result->text = "Waiting for input...";
update_label(result);

add_to_box(vbox, name_entry->entry);
add_to_box(vbox, email_entry->entry);
add_to_box(vbox, submit->button);
add_to_box(vbox, result->label);
add_to_window(win, vbox->box);

display_window(win);
close_window(win);

MainStart();
ProgramEnd();
return 0;
}

```

Installation & Compilation Guide

Prerequisites

For **WSL Ubuntu** or any **Ubuntu/Debian** system:

```

bash
# Update package list
sudo apt update

# Install GTK3 development libraries
sudo apt install libgtk-3-dev

# Install build tools

```

```
sudo apt install build-essential pkg-config
```

```
# Install additional dependencies (for images, etc.)
```

```
sudo apt install libcurl4-openssl-dev
```

Setting Up the Compiler Script

Step 1: Create the `compile.sh` script in your project directory:

```
bash
```

```
nano compile.sh
```

Step 2: Paste this content:

```
bash
```

```
#!/bin/bash
```

```
# Check if argument is provided
```

```
if [ -z "$1" ]; then
```

```
    echo "Usage: compile <main_file_without_.c>"
```

```
    echo "Example: compile myprogram"
```

```
    exit 1
```

```
fi
```

```
MAIN_FILE="$1.c"
```

```
# Fixed file (your shared library)
```

```
COMMON_FILE="structures.c"
```

```
# Output name (same as main file)
```

```
OUTPUT="$1"
```

```
# Check if main file exists
```

```
if [ ! -f "$MAIN_FILE" ]; then
```

```
    echo "✗ Error: $MAIN_FILE not found!"
```

```
    exit 1
```

```
fi
```

```
# Check if structures.c exists
```

```
if [ ! -f "$COMMON_FILE" ]; then
```

```
    echo "✗ Error: $COMMON_FILE not found!"
```

```
    exit 1
```

```
fi
```

```
echo "🔧 Compiling $MAIN_FILE..."
```

```
# Compile with all warnings enabled
```

```
gcc -Wall -Wextra "$MAIN_FILE" "$COMMON_FILE" \  
`pkg-config --cflags --libs gtk+-3.0` \  
-lcurl -lm -o "$OUTPUT"
```

```
# Check result
```

```
if [ $? -eq 0 ]; then  
    echo "✔ Compilation successful!"  
    echo "▶ Run with: ./$OUTPUT"  
else  
    echo "✗ Compilation failed"  
    exit 1  
fi
```

Step 3: Make it executable:

```
bash  
chmod +x compile.sh
```

Step 4: (Optional) Install globally:

```
bash  
sudo cp compile.sh /usr/local/bin/compile  
sudo chmod +x /usr/local/bin/compile
```

Now you can run `compile myprogram` from any directory (if installed globally).

Compiling Your Program

Method 1: Using the script (recommended)

```
bash  
# Make sure structures.c and structures.h are in the same directory  
compile myprogram  
  
# Run the compiled program  
./myprogram
```

Method 2: Manual compilation

```
bash  
gcc myprogram.c structures.c `pkg-config --cflags --libs gtk+-3.0` -lcurl -lm -o myprogram  
./myprogram
```

Method 3: With debug symbols

```
bash  
gcc -g -Wall myprogram.c structures.c `pkg-config --cflags --libs gtk+-3.0` -o myprogram
```

Project Structure

project directory should look like:

```
myproject/
├── structures.h      # Header file (required)
├── structures.c      # Implementation (required)
├── myprogram.c       # Your application
└── compile.sh       # Compiler script (optional)
```

Running on WSL Ubuntu

Since WSL doesn't have a native GUI by default, you need an X server:

Option 1: Windows X Server (recommended)

1. Install **VcXsrv** or **Xming** on Windows
2. Launch X Server (accept connections)
3. In WSL, set DISPLAY variable:

```
bash
```

```
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0
```

4. Or add to `~/.bashrc` for persistence:

```
bash
```

```
echo 'export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk "{print \$2}"):0' >> ~/.bashrc
```

Option 2: WSLg (Windows 11)

WSLg comes with built-in GUI support. Just run your program normally.

Troubleshooting

Common Issues and Solutions

Problem	Solution
gtk/gtk.h: No such file	Install GTK3: <code>sudo apt install libgtk-3-dev</code>

pkg-config: command not found	Install pkg-config: <code>sudo apt install pkg-config</code>
cannot open display:	Set DISPLAY for WSL (see above)
Segmentation fault	Check that you called <code>update_widget()</code> after setting properties
Widgets not showing	Ensure <code>display_window()</code> is called after adding widgets
Callbacks not working	Verify callback function signature matches: <code>void func(GtkWidget*, gpointer)</code>
Memory leaks	Use <code>g_free()</code> for strings you allocated

Debugging Tips

```
// Print GTK version at runtime
printf("GTK version: %d.%d.%d\n",
    gtk_get_major_version(),
    gtk_get_minor_version(),
    gtk_get_micro_version());
```

```
// Check if widget is properly created
if (widget == NULL) {
    g_print("Widget creation failed!\n");
}
```

Best Practices

1. **Always call** `update_widget()` after setting properties
2. **Use** `close_window()` to handle destroy signals
3. **Free string allocations** you make (except GTK-managed ones)
4. **Group related radio buttons** with the same `group_id`
5. **Set window dimensions** before adding content
6. **Use boxes for layout** - don't position widgets absolutely

API Quick Reference

Core Functions

```
void ProgramStart(void); // Initialize GTK
```

```
void MainStart(void);           // Start event loop
void ProgramEnd(void);         // Cleanup
```

Window Functions

```
window* create_window(void);
void update_window(window *win);
void display_window(window *win);
void close_window(window *win);
void add_to_window(window *win, GtkWidget *widget);
```

Widget Creation

```
button* create_button(void);
label* create_label(void);
entry* create_entry(void);
progress_bar* create_progress_bar(void);
spinner* create_spinner(void);
image* create_image(void);
check_button* create_check_button(void);
radio_button* create_radio_button(const char *group_id, const char *label);
```

Layout Functions

```
box* create_hbox(int spacing, gboolean homogeneous);
box* create_vbox(int spacing, gboolean homogeneous);
void add_to_box(box *bx, GtkWidget *widget);
void add_to_box_with_properties(box *bx, GtkWidget *widget,
                               gboolean expand, gboolean fill, guint padding);
```

Utility Functions

```
char* get_entry_text(entry *en);
void set_entry_text(entry *en, const char *text);
void set_progress_value(progress_bar *pb, double value);
void start_spinner(spinner *spin);
void stop_spinner(spinner *spin);
gboolean is_radio_button_active(radio_button *rdb);
gboolean is_check_button_active(check_button *chb);
```

This library provides a complete foundation for building GTK3 applications in C with a simplified, intuitive API. Happy coding! 🚀